

**МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И
МАССОВЫХ КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ
Ордена трудового Красного Знамени федеральное государственное
бюджетное
образовательное учреждение высшего образования
«Московский технический университет связи и информатики»**

Кафедра Математическая кибернетика и информационные технологии

**Лабораторная работа №6
Разработка веб-приложения на Spring Boot.
Основы тестирования
по дисциплине
«Информационные технологии и программирование»**

Выполнил: студент гр. БВТ2403
Махмудов А. А.

Руководитель:

Москва, 2026 г.

Цель работы

Изучить основы тестирования в Java и Spring Boot-приложениях: научиться писать модульные тесты, проверять поведение отдельных компонентов приложения, изолировать зависимости с помощью mock-объектов, а также познакомиться с базовыми подходами к тестированию сервисного слоя и веб-приложений.

Задачи

- Подготовить проект spring-lab3-notifications (из лабораторных №4–5) к написанию тестов.
- Изучить базовые возможности JUnit 5: аннотации, assertions, проверка исключений, методы жизненного цикла.
- Освоить библиотеку Mockito: создание моков, стабов, проверка вызовов, использование @Mock, @InjectMocks, @Spy, verify().
- Написать unit-тесты для UserService, NotificationService, AuthService.
- Протестировать успешные сценарии и ситуации с ошибками (например, пользователь не найден).
- Проверить взаимодействие с репозиториями через verify().
- Написать простой web-тест с @WebMvcTest и MockMvc.
- Выполнить самостоятельные задания и ответить на контрольные вопросы.

Ход работы

0. Подготовка

Для тестирования использовался проект spring-lab3-notifications, разработанный в предыдущих лабораторных работах. В файл pom.xml добавлена зависимость spring-boot-starter-test (обычно присутствует в проектах Spring Boot по умолчанию). При необходимости была добавлена явная зависимость на mockito-core.

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-test</artifactId>
  <scope>test</scope>
</dependency>
<dependency>
  <groupId>org.mockito</groupId>
  <artifactId>mockito-core</artifactId>
  <scope>test</scope>
</dependency>
```

Тестовые классы размещались в src/test/java в пакетах, зеркалирующих пакеты основного кода.

1. Первая структура unit-теста на JUnit

```
package org.example;
import org.junit.jupiter.api.Test; import static
org.junit.jupiter.api.Assertions.assertEquals;
public class SimpleTest
{
    @Test    void
    shouldAddNumbers() {
    int result = 2 + 3;
    assertEquals(5, result);
    }
}
```

Тест успешно выполнялся, что подтвердило корректную настройку тестового окружения.

2. Основные возможности JUnit для unit-тестов

2.1. Проверка значений

```
@Test
void shouldCheckAssertions() {
    String value = "Spring";
    assertEquals("Spring", value);
    assertNotNull(value);
    assertTrue(value.startsWith("Sp"));
    assertFalse(value.isEmpty()); }
```

2.2. Проверка исключений

```
@Test void shouldThrowException() {      assertThrows(ArithmeticException.class,
() -> {      int result = 10 / 0;
    });
}
```

2.3. Методы жизненного цикла

```
@BeforeEach
void setUp() {
    System.out.println("Подготовка перед каждым тестом");
}

@AfterEach void
tearDown() {
    System.out.println("Завершение после каждого теста"); }
```

3. Что такое unit-тест в нашем

Unit-тесты в приложении системы уведомлений были написаны для сервисного слоя (UserService, NotificationService, AuthService), так как именно там сосредоточена бизнес-логика. Репозитории и контроллеры тестируются отдельно (интеграционные тесты или @WebMvcTest).

4. Первый unit-тест с Mockito для UserService

Создан класс UserServiceTest с использованием @ExtendWith(MockitoExtension.class). Репозиторий UserRepository заменён моком, сервис внедрён через @InjectMocks.

```
@ExtendWith(MockitoExtension.class) class
UserServiceTest {
    @Mock    private UserRepository
userRepository;
    @InjectMocks
    private UserService userService;
    @Test
    void shouldCreateUser() {
```

```

        UserDto dto = UserDto.builder()
            .name("Иван Иванов")
            .email("ivan@example.com")
            .phone("+79990001122")
            .deviceToken("device-token-123")
            .telegramChatId("123456789")
            .build();
        User savedUser = new User();
        savedUser.setName(dto.getName());
        savedUser.setEmail(dto.getEmail());
        when(userRepository.save(any(User.class))).thenReturn(savedUser);
        User result = userService.createUser(dto);
        assertNotNull(result);          assertEquals("Иван
Иванов", result.getName());
        assertEquals("ivan@example.com", result.getEmail());
    }
}

```

Тест проверяет, что метод `createUser` корректно преобразует DTO в сущность и вызывает `userRepository.save()`.

5. Проверка вызова метода репозитория через `verify()`

Добавлен тест, который проверяет, что метод `save()` был вызван ровно один раз:

```

@Test
void shouldCallSaveOnRepository() {
    UserDto dto =
        UserDto.builder().name("Иван").email("ivan@example.com").build();
    when(userRepository.save(any(User.class))).thenReturn(new User());
    userService.createUser(dto);    verify(userRepository,
    times(1)).save(any(User.class));
}

```

6. Тестирование `NotificationService`

`NotificationService` зависит от двух репозиториев. Написан тест для метода `createNotification`, который мокирует `UserRepository` и `NotificationRepository`.

```

@ExtendWith(MockitoExtension.class) class
NotificationServiceTest {
    @Mock private NotificationRepository notificationRepository;
    @Mock private UserRepository userRepository;
    @InjectMocks private NotificationService notificationService;

    @Test
    void shouldCreateNotification() {
        User user = new User();
        user.setId(1L);
        user.setEmail("ivan@example.com");

        NotificationDto dto = NotificationDto.builder()
            .title("Напоминание")
            .message("Завтра занятие по Spring")
            .channel(NotificationChannel.EMAIL)
            .recipientId(1L)
            .build();

        Notification savedNotification = new Notification();
        savedNotification.setTitle(dto.getTitle());
        savedNotification.setMessage(dto.getMessage());
        savedNotification.setChannel(dto.getChannel());
        savedNotification.setRecipient(user);
        when(userRepository.findById(1L)).thenReturn(Optional.of(user));
        when(notificationRepository.save(any(Notification.class))).thenReturn(
            savedNotification);

        Notification result = notificationService.createNotification(dto);
        assertNotNull(result);
        assertEquals("Напоминание",
            result.getTitle());
        assertEquals(NotificationChannel.EMAIL,
            result.getChannel());
        assertEquals(user,
            result.getRecipient());
    }
}

```

7. Проверка исключений в сервисах

Написан негативный тест для случая, когда пользователь с указанным recipientId не найден:

```

@Test
void shouldThrowExceptionWhenUserNotFound() {
    NotificationDto dto = NotificationDto.builder()
        .title("Напоминание")
        .message("Сообщение")

```

```

        .channel(NotificationChannel.EMAIL)
        .recipientId(99L)                .build();
    when(userRepository.findById(99L)).thenReturn(Optional.empty());

    assertThrows(RuntimeException.class, () ->
notificationService.createNotification(dto));
}

```

8. Mock, Stub и Spy

Mock – объект-заглушка, имитирующий реальную зависимость.

Stub – заданное поведение мока (например, when(...).thenReturn(...)).

Spy – объект, оборачивающий реальный экземпляр, позволяющий частично переопределять методы и проверять вызовы. Пример использования spy():

```

@Test
void testSpy() {
    List<String> list = new ArrayList<>();    List<String> spyList =
spy(list);    spyList.add("Spring");
verify(spyList).add("Spring");    assertEquals(1, spyList.size());
// реальный метод add был вызван
}

```

9. Когда не нужно использовать mock-объекты

Моки не следует применять для простых value-объектов (например, DTO), а также когда тест превращается в проверку реализации, а не поведения.

Mockito рекомендуется использовать осознанно, мокируя только внешние зависимости.

10. Базовое тестирование контроллера

Создан тест для UserController с использованием @WebMvcTest и MockMvc. Зависимость UserService замочана через @MockBean.

```

@WebMvcTest(UserController.class)
class UserControllerTest {
    @Autowired private
MockMvc mockMvc;
    @MockBean private
UserService userService;

    @Test
    void shouldReturnOkForAllUsers() throws Exception {
when(userService.getAllUsers()).thenReturn(List.of());
mockMvc.perform(get("/users/all"))
        .andExpect(status().isOk());
    }
}

```

Этот тест проверяет, что эндпоинт доступен и возвращает статус 200 (без проверки содержимого).

11. Разница между unit-тестом и интеграционным тестом

Составлена таблица сравнения (в тексте отчёта):

Сессии – stateful, хранятся на сервере, используют cookie, удобны для браузерных приложений.

JWT – stateless, клиент хранит токен, подходит для REST API и мобильных приложений, требует проверки подписи.

Характеристика	Unit-тест	Интеграционный тест
Область тестирования	Один класс	Взаимодействие нескольких компонентов
Зависимости	Заменяются моками	Используются реальные (или тестовые)
Контекст Spring	Не требуется (или частично)	Обычно требуется
Скорость выполнения	Быстро	Медленнее
Пример	UserServiceTest с мокрепозиторием	Тест с @SpringBootTest и testcontainers

12. Проверка работы тестов в IntelliJ IDEA

Все тесты были запущены через IDE. Зелёная стрелка рядом с классом или методом позволяет выполнить тесты. При намеренном изменении ожидаемого значения тест падал, что подтверждает корректную работу.

Самостоятельные задания

1. Написан тест, который мокирует userRepository.findById(id) и проверяет, что возвращается правильный пользователь.


```

@Test
void shouldGetUserById() {    Long userId = 1L;    User user = new
User();    user.setId(userId);    user.setName("Иван");
when(userRepository.findById(userId)).thenReturn(Optional.of(user));
    User result = userService.getUserById(userId);
assertEquals(userId, result.getId());    assertEquals("Иван",
result.getName());
}

```

2. Unit-тест для deleteUser() с проверкой вызова userRepository.delete()

```

@Test
void shouldDeleteUser() {
Long userId = 1L;    User
user = new User();
user.setId(userId);

when(userRepository.findById(userId)).thenReturn(Optional.of(user));
userService.deleteUser(userId);    verify(userRepository,
times(1)).delete(user);
}

```

3. Тест для getNotificationById() – аналогичен пункту 1, но для NotificationService.

4. Негативный тест для случая, когда уведомление не найдено

```

@Test
void shouldThrowWhenNotificationNotFound() {    Long id =
999L;
when(notificationRepository.findById(id)).thenReturn(Optiona
l.empty());    assertThrows(RuntimeException.class, ()
-> notificationService.getNotificationById(id)); }

```

5. Тесты для AuthService - проверяют регистрацию: мокируется UserRepository, проверяется, что пароль кодируется и сохраняется.

```

@Test
void shouldRegisterUser() {
    RegisterRequest request = new RegisterRequest();
request.setName("Тест");    request.setEmail("test@example.com");
request.setPassword("secret");
when(userRepository.save(any(User.class))).thenReturn(new User());
authService.register(request);
verify(userRepository).save(any(User.class)); }

```

6. Использование verify(..., times(1)) – уже использовано в задании 2.

7. Применение `spy()` к простому списку – пример приведён в части 8.
8. `@WebMvcTest` для `NotificationController` – создан тест, аналогичный тесту для `UserController`, с проверкой статуса для эндпоинта `/notifications/all`.

Вывод

В ходе лабораторной работы были освоены основы тестирования в Spring Boot-приложениях. Изучены JUnit 5 (аннотации, assertions, проверка исключений) и Mockito (создание моков, стабов, проверка вызовов, `spy`).

Написаны unit-тесты для

сервисов `UserService`, `NotificationService` и `AuthService`, покрывающие успешные сценарии и обработку ошибок. Проверены взаимодействия с репозиториями через `verify()`. Выполнены самостоятельные задания, включая тестирование исключений и написание простого web-теста с `MockMvc` и `@WebMvcTest`.

Понимание разницы между unit-тестами и интеграционными тестами, а также правильное использование мок-объектов позволяют создавать надёжные и поддерживаемые тесты, которые ускоряют разработку и повышают качество кода.

<https://github.com/M4khmudov/ITaP/tree/main/Lab6>